Lab: Selenium Waits

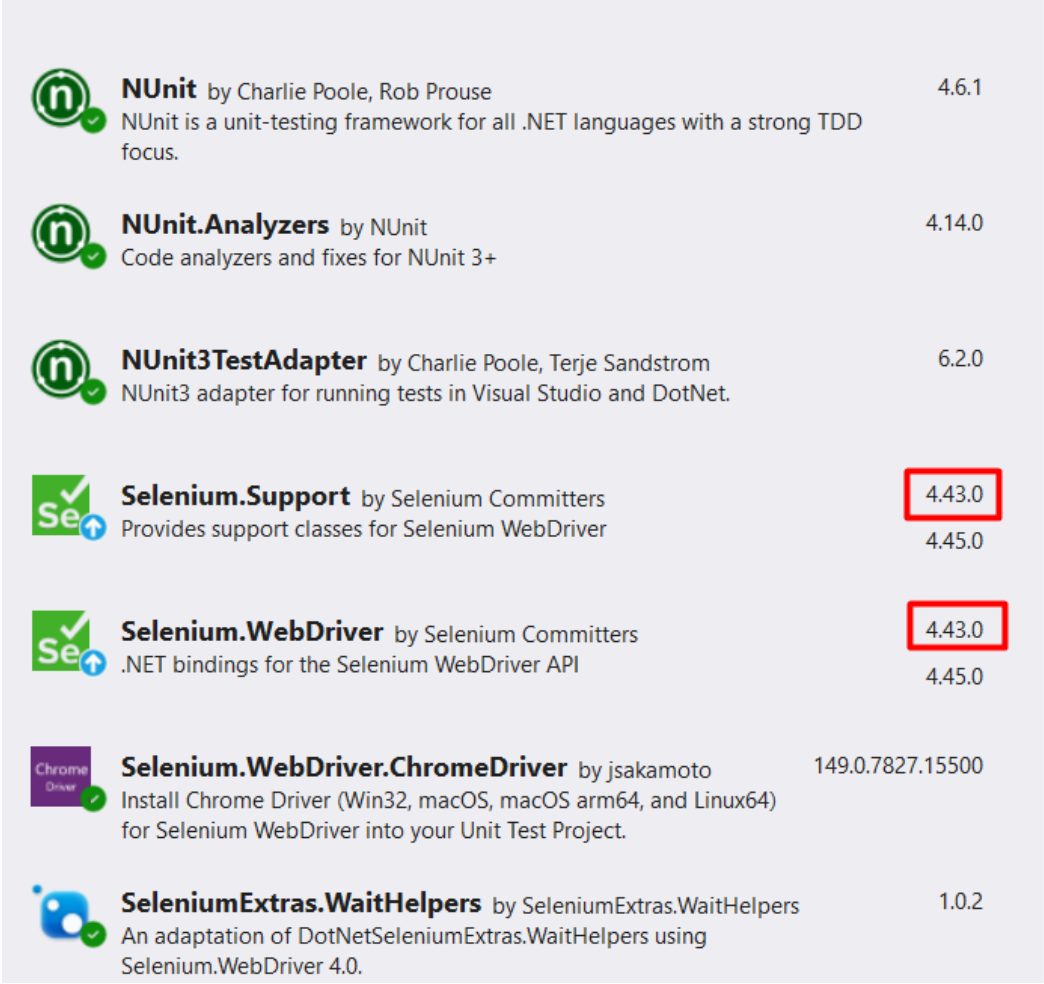
Objectives

- Observe the **consequences of not using waits**.
- Understand the **different types of waits in Selenium**.
- Learn how to **use implicit, explicit, and fluent waits** effectively.

Prerequisites

- Create a new NUnit test project.
- Install the necessary Selenium packages via NuGet:
  Selenium.WebDriver, Selenium.WebDriver.ChromeDriver, Selenium.Support.
- Initialize the ChromeDriver and navigate to the application URL.
- All the tests will be performed on the following URL:
  https://www.selenium.dev/selenium/web/dynamic.html

NOTE: The provided solutions use these nuget packages:



1. Test Without Waits

Observe failures due to elements not being immediately available.

- Open your test project and create a new test called **`AddBoxWithoutWaitsFails()`**.
- Click the button to add a new box element.
- Attempt to find the new box element.
- Assert that the new box element is displayed.

SoftUni
© SoftUni – https://softuni.org. Copyright document. Unauthorized copy, reproduction or use is not permitted.
Follow us:

```
[Test, Order(1)]
0 references
public void AddBoxWithoutWaitsFails()
{
    driver.FindElement(By.Id("adder")).Click();

    IWebElement newBox = driver.FindElement(By.Id("box0"));

    // Assert that the new box element is displayed
    Assert.IsTrue(newBox.Displayed);

}
```

- Create a new test called **`RevealInputWithoutWaitsFail().`**
- Click the button to reveal the input element.
- Attempt to find the input element.
- Interact with the input element and assert its value.

```
[Test, Order(2)]
0 references
public void RevealInputWithoutWaitsFail()
{
    // Click the button to reveal the input
    driver.FindElement(By.Id("reveal")).Click();

    // Attempt to find the revealed element (which is hidden initially)
    IWebElement revealed = driver.FindElement(By.Id("revealed"));

    // Try to interact with the revealed element
    revealed.SendKeys("Displayed");

    // Assert that the value was set correctly
    Assert.That(revealed.GetAttribute("value"), Is.EqualTo("Displayed"));

}
```

Note: These tests will fail because the elements are not ready for interaction without waits.

2. Using Thread.Sleep

- Create a new test called **`AddBoxWithThreadSleep().`**
- Click the button to add a new box element.
- Use **Thread.Sleep** to wait for a fixed amount of time (e.g., 3 seconds).
- Attempt to find the new box element after the sleep period.
- Assert that the new box element is displayed.

```
[Test, Order(3)]
0 references
public void AddBoxWithThreadSleep()
{
    // Click the button to add a box
    driver.FindElement(By.Id("adder")).Click();

    // Wait for a fixed amount of time (e.g., 3 seconds)
    Thread.Sleep(3000);

    // Attempt to find the newly added box element
    IWebElement newBox = driver.FindElement(By.Id("box0"));

    // Assert that the new box element is displayed
    Assert.That(newBox.Displayed, Is.True);
}
```

Note: While Thread.Sleep can make tests pass by introducing fixed delays, it is inefficient and can make tests slow and unreliable.

© SoftUni – https://softuni.org. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.

3. Implicit Waits

You are already familiar with Implicit Waits and have used them a couple of times, but let's practice once again.

- Create a new test method called **`AddBoxWithImplicitWait().`**
- Set an implicit wait of 10 seconds.
- Click the button to add a new box element.
- Attempt to find the new box element.
- Assert that the new box element is displayed.

```
[Test, Order(4)]
0 references
public void AddBoxWithImplicitWait()
{
    // Click the button to add a box
    driver.FindElement(By.Id("adder")).Click();

    /// Set up implicit wait
    driver.Manage().Timeouts().ImplicitWait = TimeSpan.FromSeconds(10);

    // Attempt to find the newly added box element
    IWebElement newBox = driver.FindElement(By.Id("box0"));

    // Assert that the new box element is displayed
    Assert.That(newBox.Displayed, Is.True);
}
```

Create a new test method called **`RevealInputWithImplicitWaits().`**
Set an implicit wait of 10 seconds.
Click the button to reveal the input element.
Attempt to find the input element.
Assert that the input element exists and is interactable by checking its tag name.

```
[Test, Order(5)]
0 references
public void RevealInputWithImplicitWaits()
{
    // Set up implicit wait
    driver.Manage().Timeouts().ImplicitWait = TimeSpan.FromSeconds(10);

    // Click the button to reveal the input
    driver.FindElement(By.Id("reveal")).Click();

    // Find the revealed element (implicit wait will handle the wait)
    IWebElement revealed = driver.FindElement(By.Id("revealed"));

    Assert.That(revealed.TagName, Is.EqualTo("input"));
}
```

Note: Implicit waits instruct Selenium to wait up to a specified amount of time before throwing an exception if the element is not found, improving reliability for dynamic content. This wait is set globally, meaning it applies to all elements in the WebDriver session.

4. Explicit Waits

- Create a new test called **`RevealInputWithExplicitWaits().`**
- Click the button to reveal the input element.
- Use an explicit wait to wait for the input element to be displayed.
- Interact with the input element and assert its value.

SoftUni

© SoftUni – https://softuni.org. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.

Follow us:

```
[Test, Order(6)]
0 references
public void RevealInputWithExplicitWaits()
{

    IWebElement revealed = driver.FindElement(By.Id("revealed"));
    driver.FindElement(By.Id("reveal")).Click();

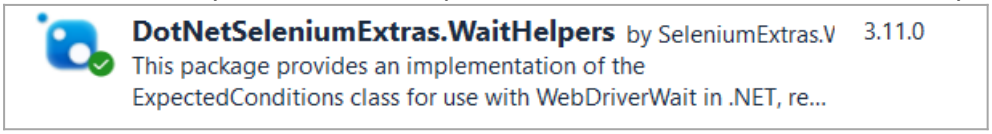
    WebDriverWait wait = new WebDriverWait(driver, TimeSpan.FromSeconds(2));
    wait.Until(d => revealed.Displayed);

    revealed.SendKeys("Displayed");

    Assert.That(revealed.GetAttribute("value"), Is.EqualTo("Displayed"));
}
```

5. Fluent Wait with Expected Conditions and Ignored Exceptions

In order to use Expected Conditions, you need to install SeleniumExtras.WaitHelpers



OR



Add it to the necessary namespaces.

```
using OpenQA.Selenium.Chrome;
using OpenQA.Selenium;
using OpenQA.Selenium.Support.UI;
using SeleniumExtras.WaitHelpers;
```

- Create a new test called **AddBoxWithFluentWaitExpectedConditionsAndIgnoredExceptions()**
- Click the button to reveal the element.
- Create a WebDriverWait instance with a timeout of 10 seconds and a polling interval of 500 milliseconds.
- Configure Fluent Wait to ignore specific exceptions.
- Use ExpectedConditions to wait until the new box element is present and visible.
- Verify that the newly added box element is displayed.

```
[Test, Order(7)]
0 references
public void AddBoxWithFluentWaitExpectedConditionsAndIgnoredExceptions()
{
    // Click the button to add a box
    driver.FindElement(By.Id("adder")).Click();

    // Set up FluentWait with ExpectedConditions
    WebDriverWait wait = new WebDriverWait(driver, TimeSpan.FromSeconds(10));
    wait.PollingInterval = TimeSpan.FromMilliseconds(500);
    wait.IgnoreExceptionTypes(typeof(NoSuchElementException));

    // Wait until the new box element is present and displayed
    IWebElement newBox = wait.Until(ExpectedConditions.ElementIsVisible(By.Id("box0")));

    // Assert that the new box element is displayed
    Assert.That(newBox.Displayed, Is.True);
}
```

SoftUni
© SoftUni – https://softuni.org. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.
Follow us:

6. Custom Waits Conditions

- Create a new test called **`RevealInputWithCustomFluentWait().`**
- Click the button to reveal the input element.
- Create a WebDriverWait instance with a timeout of 5 seconds and a polling interval of 200 milliseconds.
- Configure Fluent Wait to ignore specific exceptions.
- Use a custom wait condition to send keys to the revealed element.
    - The custom wait condition within wait.Until should be a lambda expression that tries to send keys to the revealed element. The wait should continue until the element is interactable and the keys are sent successfully.
- Verify that the revealed element is displayed and its value is set correctly.

```
[Test, Order(8)]
0 references
public void RevealInputWithCustomFluentWait()
{
    IWebElement revealed = driver.FindElement(By.Id("revealed"));
    driver.FindElement(By.Id("reveal")).Click();

    WebDriverWait wait = new WebDriverWait(driver, TimeSpan.FromSeconds(5))
    {
        PollingInterval = TimeSpan.FromMilliseconds(200),
    };
    wait.IgnoreExceptionTypes(typeof(ElementNotInteractableException));

    wait.Until(d => {
        revealed.SendKeys("Displayed");
        return true;
    });

    Assert.That(revealed.TagName, Is.EqualTo("input"));
    Assert.That(revealed.GetAttribute("value"), Is.EqualTo("Displayed"));
}
```

Note: This demonstrates how to create custom conditions that are not covered by predefined expected conditions, providing flexibility in handling unique scenarios in web automation.

7. Exceptions

Modify the First Two Tests to Assert Exceptions

- Modify the AddBoxWithoutWaitsFails test to assert the NoSuchElementException thrown when the new box element is not found.
- Modify the RevealInputWithoutWaitsFail test to assert the ElementNotInteractableException thrown when the input element is not interactable.

© SoftUni – https://softuni.org. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.